Test Adequacy via Mutation Analysis

Members:
Dhruv Patel
Vraj Patel

1. Introduction 3
1.1 Selected Projects 3
2. PITest Analysis of JSOUP 4
2.1 Project Overview 4
2.2 Project Selection Criteria 4
2.3 Running the Mutation Tests 4
2.4 Results 5
2.5 Group Analysis (Analysis of Test Suite) 7
2.6 Individual Analysis 9
3. Stryker Analysis of Polly C# 21
3.1 Project Overview 21
3.2 Project Selection Criteria 21
3.3 Setup and Environment 21
3.4 Running the Mutation Tests 21
3.5 Group Analysis 22
3.6 Individual Analysis 24
4. Comparative Analysis (and Conclusion) 34

1. Introduction

Mutation analysis is a crucial technique used in software testing to evaluate the efficiency and effectiveness of test suites. This assignment aims to perform mutation analysis on existing test suites of two open-source projects. By introducing mutations into the source code and observing whether the existing tests can detect these changes, we can gain valuable insights into the strengths and weaknesses of these projects.

Our objectives for this assignment are:

- To access the comprehensiveness of existing test suites in real-world open-source projects.
- To identify any potential weak points in test coverage and design.
- To compare different mutation testing frameworks across various programming languages (Java and C# in our case).
- To gain practical knowledge in applying mutation testing techniques to substantial codebases.

1.1 Selected Projects

For this assignment, we have chosen two different open-source projects (more details in sections 2 and 3):

1. JSOUP (Java): A popular HTML parsing library in Java, which we analyzed using **PITest**.
    - Openhub URL: https://openhub.net/p/jsoup
    - GitHub URL: https://github.com/jhy/jsoup
    - Test Framework: **JUnit 5**
2. Polly (C#): A .NET resilience and transient-fault-handling library, which we analyzed using **Stryker**.
    - Openhub URL: https://openhub.net/p/App-vNext-Polly
    - GitHub URL: https://github.com/App-vNext/Polly
    - Test Framework: **xUnit**

We selected these projects based on requirements as per the assignment question which includes more than 5000 lines of code and a comprehensive existing testing suite making them ideal for mutation analysis.

We tested several projects to ensure they ran well on our VM without compilation errors. We found most open-source projects outdated or without active development/maintenance. Hence, it was important for us to ensure the project we chose would conduct mutation analysis accurately.

2. PITest Analysis of JSoup

2.1 Project Overview

Jsoup is a Java tool that simplifies working with real-world HTML and XML. It offers an easy-to-use API for URL fetching, data parsing, extraction, and manipulation using DOM API methods, CSS, and XPath selectors. Jsoup implements the HTML5 specification and parses HTML to the same DOM as modern browsers.

The tool scrapes and parses HTML from various sources, finds and extracts data using DOM traversal or CSS selectors, manipulates HTML elements, attributes, and text, cleans user-submitted content against a safelist to prevent XSS attacks and outputs tidy HTML.

(Reference: https://jsoup.org/)

2.2 Project Selection Criteria

Below is the list of criteria we identified from the project:

- **Size of the code base**: 32.7k lines of code
- **Operating System**: Ubuntu 22.04.4 LTS (set as a virtual machine through WSL)
- **Language**: Java 11
- **Build tool**: Maven
- **Build time to compile and link an executable from source code**: 2m39.994s (using mvn clean install)
- **Test Suit Infrastructure**: JUnit 5
- **Number of test files**: 73 Java test files
- **Number of tests**: Test run is 1364 unit tests and 45 skipped tests
- **Lines of test code**: 15,437 lines (using cloc src/test/java/)
- **Execution time for the entire suite**: 0m22.213s (using mvn test)

2.3 Running the Mutation Tests

We configured PITest in the JSoup project's pom.xml file by adding the following plugin configuration:

```xml
<plugin>
   <groupId>org.pitest</groupId>
   <artifactId>pitest-maven</artifactId>
   <version>1.8.1</version>
   <dependencies>
      <dependency>
         <groupId>org.pitest</groupId>
         <artifactId>pitest-junit5-plugin</artifactId>
         <version>0.12</version>
      </dependency>
   </dependencies>
   <configuration>
      <targetClasses>org.jsoup.*</targetClasses>
      <targetTests>org.jsoup.*</targetTests>
      <mutators>ALL</mutators>
      <verbose>true</verbose>
      <timeoutConstant>120000</timeoutConstant>
      <timeoutFactor>2</timeoutFactor>
      <outputFormats>
         <param>HTML</param>
      </outputFormats>
   </configuration>
</plugin>
```

(We learned how to program this from here)

Initially, we did not specify any custom mutation operators, allowing us to use all available options. For our group analysis, it would be important to test the tool rigorously. However, for individual analysis, we did choose specific classes and mutation operators based on the overall report generated to focus on the most important parts that would give greater results.

During our learning, we realized the project occasionally timed out during mutation testing, hence we decided to increase the timeout in the configuration. As a result, this took significantly longer than expected as seen below (28 hours and 49 minutes):

```
================================================================================
================================================================================
- Statistics
================================================================================
>> Line Coverage: 8385/9340 (90%)
>> Generated 22968 mutations Killed 16784 (73%)
>> Mutations with no coverage 2147. Test strength 81%
>> Ran 678183 tests (29.53 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1 d 04:49 h
[INFO] Finished at: 2024-10-20T06:40:50-07:00
[INFO] ------------------------------------------------------------------------
```

2.4 Results

Below are the results we got from the coverage report:

- **Number of Classes**: 67
- **Line Coverage**: 90% (8317 / 9261)

- **Mutation Coverage**: 73% (16,784 / 22,968)
- **Test Strength**: 81% (16,784 / 20,821)
- **Surviving Mutants**:

Package	Total Mutants	Surviving Mutants	Survival Rate
org.jsoup	189	60	31.75%
org.jsoup.helper	2730	702	25.71%
org.jsoup.internal	991	227	22.91%
org.jsoup.nodes	5451	815	14.95%
org.jsoup.parser	9760	3392	34.75%
org.jsoup.safety	807	123	15.24%
org.jsoup.select	2737	562	20.53%

Average Survival Rate: 23.69%

The high line coverage shows that the test suite executes a significant portion of the code. Additionally, the mutation coverage does show the test suite is effective in detecting many code changes. However, there is improvement still needed in catching more subtle mutations. Overall, the test strength is highly effective as well to show us the ratio of killed mutants that were not equivalent. Some packages do show high coverage and mutation detection rates as shown in the table above. However, the classes that have the lowest mutation coverage will need more analysis which is done individually (in section 2.2). Below is the summary:

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
67	90%	8317/9261	73%	16784/22968	81%	16784/20821

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
org.jsoup	5	77%	55/71	68%	129/189	96%	129/135
org.jsoup.examples	3	0%	0/100	0%	0/303	0%	0/0
org.jsoup.helper	10	89%	1061/1192	74%	2028/2730	83%	2028/2442
org.jsoup.internal	6	91%	299/329	77%	764/991	83%	764/917
org.jsoup.nodes	17	96%	1804/1874	85%	4636/5451	89%	4636/5233
org.jsoup.parser	15	88%	3844/4378	65%	6368/9760	72%	6368/8846
org.jsoup.safety	2	100%	295/295	85%	684/807	86%	684/799
org.jsoup.select	9	94%	959/1022	79%	2175/2737	89%	2175/2449

2.5 Group Analysis (Analysis of Test Suite)

2.5.1 Strengths and Weaknesses

Strengths:

- The test suite does demonstrate a high level of coverage which would not affect the tool's main functionality. It effectively kills most mutants related to simple removals of statements, basic return value changes, or mathematical changes.
- Mutants related to error handling were consistently killed as well showing that the suite is well programmed for common scenarios.
- The main classes such as the ones inside the parser show a strong mutation coverage that reflects the key parts of the project.

Weaknesses:

- Most of the surviving mutants are related to edge cases such as null handling (e,g return null), and overflows. The test suite does not fully handle these cases which might introduce bugs.
- Some classes ran into memory errors indicating limited ability to thoroughly test the entire codebase. This is a significant challenge in testing large code blocks.
- Special configuration was needed to tackle runtime errors.

2.5.2 Improvements

- Based on the weaknesses, a major improvement would be to strengthen the error handling part, where exceptions should be thrown if wrong inputs are passed.
- JSoup should incorporate mutation testing into regular development to have regular functionality rather than just depending entirely on its test suite.
- Existing tests need to be reviewed and refactored where mutants survived. Complex methods need to be introduced that would include assertions for unexpected conditions to improve code quality.
- The main focus should be on high-coverage by prioritizing areas with the highest potential for bugs.

2.5.3 Costs and Benefits of Mutation Analysis

Costs:

- Mutation testing is extremely time-consuming compared to other testing techniques. A full test run of JSoup took more than 24 hours. In reality, projects with a greater line coverage would take way more. However, to a certainty, it depends on the system being used as well. For example, my VM has 8 threads which is moderate.

- In some cases, manual effort is required to identify and address surviving mutants. For example, as seen in section 3, particularly distinguishing equivalent mutants from valid ones, requires significant manual effort.
- Runtime errors, memory errors, and timeouts are common with large-scale mutation testing projects which would need configuration changes.

Benefits:

- The report generated by PITest provides a detailed evaluation of the test suite's performance compared to other metrics. It does highlight the specific weaknesses and areas where cases were not covered.
- The reports also did show NO_COVERAGE indicating there are mutants in the code that were not executed by any of the tests in the test suite. This would help determine which test cases need to be added.

2.5.4 Applying PITest to JSoup

Setting up PITest for JSoup was relatively straightforward due to the documentation provided by PITest. Since JSoup already used JUnit as its testing framework, configuring it was simple. With Maven integrating PITest directly into the project required minimal changes to the existing build configuration. The only difficult part was figuring out that the right versions were used for compatibility. For example, for this particular project, only Java 11 with JUnit 5 worked. Initially, I tried using Java 23 which would not allow the project to compile.

Analyzing the mutation report was way more time-consuming than the initial setup but overall PITest was effective and easy to use.

2.5.6 Obstacles and Solutions

When applying PITest to JSoup, the mutation testing process encountered two major issues:

- Memory Errors: A few classes, like the parser class, ran into memory errors during the testing process. These errors were not significant, affecting only a small percentage of the tests (0.122%).
    - Memory Error = 12/9760 = 0.122%
- Runtime Errors: Several classes encountered runtime errors during mutation testing, representing a larger percentage of the total mutations (3.75%). For example, the parse class:
    - Run Error = 366/9760 = 3.75%

Solutions:

For memory errors, since the percentage was low (less than 1%), they were ignored, and only runtime errors were addressed by increasing the timeout factor for individual classes that were to be individually analyzed (in Section 2.5).

To tackle the runtime errors, we increased the timeout factor in the pom.xml to 2 (from the default of 1.25), which allowed more time for mutation tests. This would mean the mutation test would allow doubling the time compared to the normal time it would take for that particular test to run. Running the mutation analysis on individual classes did solve most of these runtime errors.

Instead of re-running the entire test suite, we focused on running tests for the three individual classes which reduced the overall time and complexity needed to rerun the tests.

2.5.7 Insights

Based on the overall analysis, we observed the following:

- Test Suite gaps are hard to understand only with code coverage. Despite high code coverage, there is a limitation of relying solely on line or branch coverage metrics as these do not necessarily reflect the thoroughness of the test cases in handling several paths.
- However, mutation analysis does help in writing stronger tests. The surviving mutants did allow us to think about how certain tests could be written that were probably not considered. However, since we do not understand JSoup in great detail, it would be hard to say this with accuracy.
- We gained insights into how real-world development environment would look like where there are unusual conditions. Particularly for JSoup which is widely used.
- Tools like PITest integrate well with other pipelines making the automation of mutation testing as part of regular development much easier improving long-term code quality.

2.6 Individual Analysis

Due to the project's complexity, we focused on three classes individually for greater insights. We picked the following three classes for analysis due to various reasons including low mutation scores to understand the test suite:

- HtmlTreeBuilderState.java (org.jsoup.parser)
- HttpConnection.java (org.jsoup.helper)
- SimpleBufferedInput.java (org.jsoup.internal)

For these methods, we focused on the most relevant mutation operators based on the documentation here. For each class, we picked 10 mutants (both survived and killed) to get some insights into how these changes affect the tool.

(Please note, that for such an analysis, we decided to only focus on 10 generated mutations that would give the greatest insights. To do this, we modified our pom.xml to include a list of non-experimental mutation operators (as the original report with "ALL enabled" generated experimental ones as well) to give just the few tests we needed for analysis. Hence, the mutation scores in the sections below are for the individual classes based on the modifications.)

Below is the list of the most relevant operators we used:

```
<mutators>
  <mutator>CONDITIONALS_BOUNDARY</mutator>
  <mutator>INCREMENTS</mutator>
  <mutator>INVERT_NEGS</mutator>
  <mutator>MATH</mutator>
  <mutator>NEGATE_CONDITIONALS</mutator>
  <mutator>RETURN_VALS</mutator>
  <mutator>VOID_METHOD_CALLS</mutator>
  <mutator>EMPTY_RETURNS</mutator>
  <mutator>FALSE_RETURNS</mutator>
  <mutator>TRUE_RETURNS</mutator>
  <mutator>NULL_RETURNS</mutator>
  <mutator>PRIMITIVE_RETURNS</mutator>
  <mutator>REMOVE_CONDITIONALS_EQUAL_ELSE</mutator>
</mutators>
```

For a description of each of these mutation operators, refer here.

2.6.1 HtmlTreeBuilderState.java (org.jsoup.parser)

The HtmlTreeBuilderState.java in jsoup's parser library mainly represents the current state of the HTML Tree Builder during parsing. Each state focuses on the processing logic of that particular state of HTML parsing. The class focuses on the specific rules of HTML5 to handle different scenarios and edge cases.

Here are the results of the class:

Line Coverage	Mutation Coverage	Test Strength
87% (1116/1283)	51% (1663/3257)	57% (1663/2920)

Below is the table of mutants generated for the HtmlTreeBuilderState class:

Mutation Operator	Original Code	Mutated Code	Killed/Survived
NEGATE_CONDITIONALS	else if (name.equals("tr"))	} else if (!name.equals("tr")) {	KILLED
REMOVE_CONDITIONALS_EQUAL_ELSE	else if (name.equals("tr"))	} else if (false) {	KILLED
VOID_METHOD_CALLS	tb.transition(InTableBody);	(Removed)	KILLED
REMOVE_CONDITIONALS_EQUAL_ELSE	else if (t.isEOF())	else if (false)	KILLED
MATH	int i = stack.size() - 1;	int i = stack.size() - 1;	KILLED
NEGATE_CONDITIONALS	if (tb.getFromStack("svg") == null)	if (tb.getFromStack("svg") != null)	KILLED
VOID_METHOD_CALLS (1)	tb.framesetOk(false);	(Removed)	SURVIVED
NEGATE_CONDITIONALS (2)	if (el.nameIs("li"))	if (!el.nameIs("li"))	SURVIVED
RETURN_VALS (3)	return tb.process(t, InHead);	return (tb.process(t, InHead) == 0 ? 1 : 0);	SURVIVED
FALSE_RETURNS (4)	return inBodyStartTag();	return false;	SURVIVED

Below is the list of test cases for the surviving cases in the same order as the table above:

1	**Test:** The test checks the behaviour of the tb object before and after the call. Add assertions **Test Input**: Provide with an HTML tag that will trigger the framesetOk(false) **Oracle**: After the call, the test should assert saying it is not allowed
2	**Test**: The test should handle cases where the element's name is "li" **Test Input**: Provide an HTML tag with list items (<li>) with both valid and invalid formats. **Oracle**: After the call, the test should assert to check elements with "li" are processed correctly.
3	**Test**: The test should check for the right return from tb.process() and also ensure edge cases are handled. **Test Input**: Provide with an HTML tag that will trigger the InHead element. **Oracle**: After the call, it should assert and return false or a valid value showing it is handled rightly.
4	**Test**: The test should check if the correct boolean value is returned. **Test Input**: Provide an HTML tag with valid tags in the body with unexpected cases as well like closing start tags. **Oracle**: It should assert that it returns false for improper tags and true if valid.

To provide a detailed analysis, I answered the following questions from the table above:

1) How many mutants are killed? How many mutants survived?
    - 1663 mutants were killed.
    - 1595 mutants survived.
2) Are there any duplicate mutants?
    - After reviewing the 10 mutants, none of them are exact duplicates in the selected class. Each one that we picked had a different change in the code. However, there were other cases like dealing with several return values in the class, particularly with boolean that showed repeated weakness.
3) What are the challenges involved?
    - Unexpected HTML tags may cause false positives. The tests may pass but it would not provide the overall effectiveness of the test suite.
4) Calculate and report the mutation score/effectiveness for your particular mutants.
    - Killed Mutants/Total Mutants x 100% = 537/1633 x 100 = 32.89%

5) Does it affect the results?
    - Yes, the low mutation score significantly affects the results. A mutation score of 32.89% for the entire class shows us that the test suite does not cover many important scenarios, particularly those involving return values and conditional logic. The surviving mutants highlight the gap with the current test coverage to handle edge cases and non-standard parsing cases. Additionally, particularly for complex HTML documents, these surviving mutants indicate potential real-world bugs that could lead to unexpected behaviour which the test suite does not currently address.
6) What do these results say about the effectiveness of the test suite?
    - While the test suite does cover basic functionality, such as common transitions and arithmetic operations, it does miss critical edge cases. The low mutation score shows how the test suite lacks comprehensive tests for handling usual or malformed HTML documents, as well as elements that are less commonly used i n parsing. To improve the effectiveness of the test suite, the test suite should focus on asserting the correct return values, testing various conditional follows, including nested elements, and expanding the test coverage to ensure all possible scenarios and transitions are thoroughly tested.
7) Are there any cases with No Coverage?
    - There are 339 cases of No Coverage in the report. We ignored this as there are no test cases to verify this. This is a moderate number to add test cases.

2.6.2 HttpConnection.java (org.jsoup.helper)

This class implements the connection interface which is used for making HTTP requests in jsoup. The class allows configuration of various aspects of the HTTP request such as setting the URL, setting the request method, setting request headers, adding request data for POST requests, configuring proxies, and much more. It manages both the request and response aspects of the HTTP connection.

(Reference: here)

Here are the results of the class:

Line Coverage	Mutation Coverage	Test Strength
88% (580/658)	69% (993/1444)	79% (993/1254)

Below is the table of mutants generated for the class:

Mutation Operator	Original Code	Mutated Code	Killed/Survived
VOID_METHOD_CALLS **(1)**	Validate.notEmptyParam(url, "url");	Removed the call to Validate::notEmptyParam	SURVIVED
NULL_RETURNS	return this;	Replaced return value with null	KILLED
MATH	String value = keyvals[i+1];	String value = keyvals[i-1];	KILLED
RETURN_VALS	return keyVal;	if (x != null) null else throw new RuntimeException	KILLED
REMOVE_CONDITIONALS_EQUAL_ELSE **(2)**	if (input.length >= 3)	if (false)	SURVIVED
RETURN_VALS **(3)**	return (T) this;	if (x != null) null else throw new RuntimeException	SURVIVED
VOID_METHOD_CALLS	w.write(URLEncoder.encode(keyVal.value(), req.postDataCharset()));	Removed call to BufferedWriter::write	KILLED
NEGATE_CONDITIONALS	return stream != null;	return stream == null;	KILLED
CONDITIONALS_BOUNDARY	if (vals.size() > 0)	Changed conditional boundary	KILLED
RETURN_VALS	return res.parse();	if (x != null) null else throw new RuntimeException	KILLED

Below is the list of test cases for the surviving cases in the same order as the table above:

1	**Test**: Write a test case to check for an empty or null url to the object and checks if a ValidationException is thrown. **Test Input**: Pass an empty or null url to the HttpConnection object. **Oracle**: The test should expect a ValidationException to be thrown when url is empty or null, verifying that the call to Validate.notEmptyParam() is required.
2	**Test**: Write a test case that will check the input array with fewer than 3 elements to ensure it is handled corrected if it has a short input. **Test Input**: Pass an input array with 1 or 2 elements. **Oracle**: The test should check whether the method correctly handles arrays with less than 3 elements, otherwise throw an error.
3	**Test**: Write a test case to check for the correct return of this object. **Test Input**: Pass a case where the header method in the class should return the current object (this) **Oracle**: This should assert that this is returned correctly and doesn't throw unexpected exceptions.

I decided to use test cases instead of equivalent mutants for these three cases because:

- For 1) the removal of the call changes the program's behaviour. Without the call, the program will no longer be able to check of the url is empty, which is important for ensuring a valid input. Hence, a test case is needed to kill this mutant.
- For 2) The mutation changes the behaviour of the method to always skip the logic that depends on input.length >= 3. It changes the behaviour of the method which might introduce bugs when handling arrays with less than 3 elements. Adding a test case will let us know the issue.
- For 3) The mutation changes to the return value which obviously affects the class. It is expected to return the this object but instead it changes to return null or throw an exception. A test case is needed to verify the correct return value.

To provide a detailed analysis, I answered the following questions from the table above:

1) How many mutants are killed? How many mutants survived?
    - Out of the total mutants, 993 mutants were successfully killed, while 451 mutants survived. This indicates that while the test suite covers many of the fundamental cases, there are several edge cases that are not handled, allowing a significant number of mutants to survive.

2) Are there any duplicate mutants?
    - After looking at all the mutants generated, including the ones not analysed in the table above, I did not find an exact duplicate mutant. However, I found similar patterns, especially in the methods 'Validate.notEmptyParam' and 'Validate.notNullParam' and return value mutations. These repeated mutations suggested that the current test does not have the capabilities to detect changes and does not ensure the correct return values.
3) What are the challenges involved?
    - The most important challenge for me was to find equivalent mutants to check whether some of the surviving mutants are actually equivalent. In cases where the validation method were methods or return values were changes, it does not immediately cause a test to fail. It is likely that the test does not have coverage for empty or null parameters. Manually understanding the mutation for equivalent would require a greater understanding of JSoup which we as a group and individually did not have. Hence, developers with in-depth knowledge would be needed which is a significant challenge for an open-source platform.
4) Does it affect the results?
    - Yes, the surviving mutants significantly affect the results. The 20 surviving mutants suggest the test suite has crucial gaps, particularly around input validation where removal of several methods survived and change of return values, often replaced with null by PITest.
5) Calculate and report the mutation score/effectiveness for your particular mutants.
    - Mutation Score = Killed Mutants/Total Mutants x 100 = 30/50 x 100 = 60%
        - While this score is significantly reasonable, the remaining 40% indicates several scenarios not covered.
6) What do these results say about the effectiveness of the test suite?
    - While providing a solid foundation, there's significant potential for enhancing robustness. The discrepancy between line coverage (88%) and mutation score (60%) highlights that coverage alone is not a sufficient measure of test quality.
    - While the test suite provides decent coverage and catches many issues, the mutation testing results reveal important gaps in its effectiveness. Focusing on the areas highlighted by the surviving mutants, particularly around validation, edge cases, and error handling, would likely lead to a more comprehensive and reliable test suite, ultimately improving the overall quality and reliability.
7) Are there any cases with No Coverage?
    - We had 190 mutants with No Coverage. We did not focus on analysing this part as it would not give insights into the test suite.

2.6.3 SimpleBufferedInput.java (org.jsoup.internal)

This class is responsible for providing input functionality for parsing HTML content. With limited knowledge of the tool, I was not able to find out the exact usage in detail. However, I chose the class since it had the lowest Mutation Coverage which would help me understand the report in depth.

(Reference: here)

Here are the results of the class:

Line Coverage	Mutation Coverage	Test Strength
81% (62/77)	50% (99/197)	60% (99/164)

Below is the table of mutants generated for the class:

Mutation Operator	Original Code	Mutated Code	Killed/Survived
MATH (1)	if (offset < 0 || desiredLen < 0 || desiredLen > dest.length - offset)	if (offset < 0 || desiredLen < 0 || desiredLen > dest.length + offset)	SURVIVED
REMOVE_CONDITIONALS_EQUAL_ELSE (2)	if (!inReadFully && bufMark < 0)	if (false)	SURVIVED
NEGATE_CONDITIONALS	if (bufAvail <= 0)	if (bufAvail > 0)	KILLED
REMOVE_CONDITIONALS_EQUAL_ELSE (3)	if (read == -1)	if (false)	SURVIVED

VOID_METHOD_CALLS	System.arraycopy(getBuf(), bufPos, dest, offset, read);	Removed call to System.arraycopy	KILLED
VOID_METHOD_CALLS (4)	super.close();	Removed call to FilterInputStream::close	SURVIVED
REMOVE_CONDITIONALS_EQUAL_ELSE	if (byteBuf == null) return;	if (false)	KILLED
NULL_RETURNS	SoftPool<>(() -> new byte[BufferSize])	Replaced return value with null	KILLED
PRIMITIVE_RETURNS	return read;	return (x == 0 ? 1 : 0)	KILLED
TRUE_RETURNS	return inReadFully;	return true;	KILLED

Below is a possible equivalent mutant for (1):

1	**Original Code:** if (offset < 0 || desiredLen < 0 || desiredLen > dest.length - offset) **Equivalent mutant**: if (offset <= 0 || desiredLen < 0 || desiredLen >= dest.length - offset) **Reason**: The mutation changes the > (greater than) comparison to >= (greater than or equal to). This could be considered if "desiredLen == dest.length - offset" does not introduce a logical error. Hence, it would be considered equivalent if both cases are handled identically.

Below is the list of test cases for the surviving cases in the same order as the table above:

2	**Test**: Add a test case where inReadFully is false and bufMark is negative to test the logic **Test Input**: inReadFully would be set to false and bufMark would be less than zero **Oracle**: This will assert that the method passes correctly and does not bypass any logic that relies on the condition.
3	**Test**: Add a test case where read would be set to different integers to test edge cases. **Test Input**: Pass read == -1 along with other integers to ensure end of the file has been reached. **Oracle**: This will assert to ensure it correctly handles the case and does not skip any logic.
4	**Test**: Add a test case that checks resources are closed properly when the function is called. **Test Input**: Open a file, perform some operations and close it. **Oracle**: This should assert if the file was properly closed and changes were saved.

To provide a detailed analysis, I answered the following questions from the table above:

1) How many mutants are killed? How many mutants survived?
    - Out of the 197 mutants generated, 99 mutants were successfully killed by the test suite, while 98 mutants survived. This reflects that although some aspects of the code are well-tested, the test suite misses almost 50% of the cases which is a significant number.
2) Are there any duplicate mutants?
    - Yes, there are duplicate mutants that I found. For example:
        - if (readlimit > BufferSize); and static final SoftPool<byte[]> BufferPool = new SoftPool<>(() -> new byte[BufferSize]);
            - Substituted 8192 with 8193 and both survived.
        - private int bufMark = -1; bufMark = -1; and if (read == -1)
            - Substituted -1 with 0 and all of them survived.
        - private boolean inReadFully = false; and return 0;
            - Substituted 0 with 1 and both survived.
    - From these examples, we can see that all the duplicate mutants from the class have survived showing a weak test coverage for edge cases.

3) What are the challenges involved?
    - The main challenge I noticed was with buffer management. Often the surviving mutants were related to buffer boundaries, management of functions like open/close, and handling certain conditions.
    - Ensuring that all these cases are covered would require detailed knowledge of how input/output stream works under different cases and writing these test cases would be complex and time-consuming. Additionally, certain surviving mutants are equivalent meaning they can not be killed. Hence, this is a difficult and manual process which can slow down the analysis.
4) Calculate and report the mutation score/effectiveness for your particular mutants.
    - Mutation Score = Killed Mutants/Total Mutants x 100 = 99/197 x 100 = 50.25%
    - This show a 50:50 ratio of the test effectiveness showing a huge gap in the test cases for this class.
5) Does it affect the results?
    - The low mutation score suggests the test suite is not thoroughly programmed to test certain conditions as the ones I mentioned before. As seen before, there are bugs that fail to be caught such as the substitutions (e.g -1 to 0, 8192 to 9193 and 0 to 1).
6) What do these results say about the effectiveness of the test suite?
    - There is plenty of room for improvement where additional tests need to be written. The presence of duplicate mutants that survived also highlights a lack of robustness in the test coverage. While some aspects of the code are tested, the test suite needs improvement to cover all scenarios, especially those involving critical conditions.
7) Are there any cases with No Coverage?
    - 33 cases of No Coverage were reported in this class. As mentioned before, this was not analyzed due to no test cases to verify.

3. Stryker Analysis of Polly C#

3.1 Project Overview

Polly provides a set of resilience and transient-fault-handling strategies that developers can easily implement in their C# applications. These strategies help manage failures, timeouts, and other unexpected behaviours in a more controlled manner.

Polly offers a comprehensive set of built-in resilience strategies including Retry, Circuit Breaker, Timeout, Fallback, Rate Limiter, Bulkhead Isolation, Cache, and Hedging, allowing developers to easily implement robust fault-handling mechanisms in their .NET applications.

(https://www.thepollyproject.org/)

3.2 Project Selection Criteria

Below is the list of criteria we identified from the project:

- **Codebase Size**: 68,715 lines of code.
- **Language:** C#
- **Test Suite Infrastructure:** xUnit
- **Number of Tests:** 2,769 test cases.
- **Lines of Code in Test Suite:** Approximately 8,000 lines.

3.3 Setup and Environment

- **Platform**: .NET-based for C# projects.
- **Build Setup**: dotnet (for C#).
- **Stryker Setup**:
    - dotnet tool install -g dotnet-stryker
    - dotnet build
    - dotnet-stryker
- **Evaluation Platform:** Ubuntu 22.04.4 LTS (set as a virtual machine through WSL), .NET version 6.0 and 8.0

3.4 Running the Mutation Tests

For the Polly project, we used **Stryker.NET** to perform mutation testing. We let Stryker.NET operate with predefined mutation operators to make it simple to use with no additional configuration for mutators.

The list of mutators that Stryker.NET used can be found here.

Unlike PITest, Stryker did not use any experimental operators which was a significant advantage.

Stryker scanned the project, generated mutants, ran the tests for each mutant, and provided a detailed mutation report.

The entire process took **1 hour and 54 minutes**, with the terminal providing real-time feedback on the test execution and the final results, as seen below:

```
[17:41:21 INF] Safe Mode! Stryker will flag mutations in Implementation as compile error.
[17:41:23 INF] 5178 mutants created
[17:41:23 INF] Capture mutant coverage using 'CoverageBasedTest' mode.
Hint: by passing "--open-report or -o" the report will open automatically and update the report in real-time.
[17:43:41 INF] 285   mutants got status CompileError. Reason: Mutant caused compile errors
[17:43:41 INF] 35    mutants got status Ignored.      Reason: no means to test this
[17:43:41 INF] 2     mutants got status Ignored.      Reason: no means to test this, stryker changes '<' to '<=' where 0 is already covered in the branch above
[17:43:41 INF] 455   mutants got status NoCoverage.   Reason: Not covered by any test.
[17:43:41 INF] 1639  mutants got status Ignored.      Reason: Removed by block already covered filter
[17:43:41 INF] 45    mutants got status Ignored.      Reason: Removed by exclude from code coverage filter
[17:43:41 INF] 2461  total mutants are skipped for the above mentioned reasons
[17:43:41 INF] 2717  total mutants will be tested
100.00% | Testing mutant 2717 / 2717 | K 2131 | S 501 | T 85 | ~0m 00s |                                                    01:54:53
```

3.5 Group Analysis

3.5.1 Results

Below are the results we got from the coverage report:

2211 | 508 | 453

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	69.70	81.32	2117	508	94	453	1721	0	285	**2211**	**961**	**5178**
src	69.93	81.65	2088	490	93	448	1701	0	278	**2181**	**938**	**5098**
test/Polly.TestUtils	56.60	62.50	29	18	1	5	20	0	7	**30**	**23**	**80**

- **Line Coverage**: In the Stryker analysis, the **line coverage** was **81.32%**, showing that a large part of the codebase is executed by the current test suite.
- Mutation Coverage:
  - **Killed Mutants:** 2,117 out of 5,178 generated mutants.
  - **Mutation Score:** 69.70%, indicating that most mutants were killed but there are areas that need stronger test coverage.
- Surviving Mutants:
  - **Total Survived Mutants:** 508
    - The surviving mutants highlight weaknesses in the test suite, particularly in handling edge cases, complex logical conditions, and rare resilience scenarios. These areas need more focused testing.

- Test Strength:
    - Strengths:
        - The test suite is effective at covering core functionalities like retry and circuit breaker policies, shown by the high covered mutation score.
    - Weaknesses:
        - Surviving and no-coverage mutants point to missing tests for certain logical paths and boundary conditions, suggesting the need for additional test cases targeting these areas.

3.5. Analysis of Test Suite

3.5.1 Strengths and Weaknesses

Strengths

- The test suite has good coverage of core logic, especially for common operations in retry and circuit breaker patterns. This was evident from the high covered mutation score (81.32%).

Weaknesses

- Identified Gaps in Coverage:
    - The uncovered and surviving mutants indicate missing test cases, especially for boundary conditions, rare resilience paths, and fallback handling.
- Proposed New Tests:
    - Add tests targeting uncovered branches and edge-case scenarios in the resilience logic, like unusual combinations of retry and circuit breaker policies.

3.5.2 Cost and Benefit

Using Stryker vs. Other Coverage Tools:

- Benefit**:** Stryker provides deeper insights by creating mutants that simulate real-world errors, which normal code coverage tools cannot do. It helps find gaps in test cases that standard code coverage might not catch.
- Cost**:** Stryker's setup and analysis take more time compared to tools depending on the functionalities of the project, making it more resource-intensive. For Polly, it took us several hours to install all the right dependencies like .NET 6.0 and 8.0 to ensure it was built with no errors first.
- Runtime Complexity**:** Stryker's longer runtimes (due to sequential mutation testing) can be challenging when analyzing larger projects.

3.5.3 Challenges

- Stryker's mutation analysis runs sequentially, resulting in longer runtimes. Adjusting the settings helped speed it up, but it was still slower compared to other tools but faster than PITest.
- Initial integration with xUnit required specific configuration changes to align with Stryker's mutation operators which needs a lot of understanding.
- Identifying and separating equivalent mutants required manual checking, which was time-consuming but essential for accurate results.

3.6 Individual Analysis

Due to the project's complexity, we focused on three classes individually for greater insights. We picked the following three classes for analysis:

- ObjectPool.cs
- PolicyBuilder.OrSyntax.cs
- AsyncTimeoutEngine.cs

3.6.1 ObjectPool.cs

Here are the results of the class:

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
ObjectPool.cs	88.24	88.24	14	2	1	0	11	0	0	15	2	28

Below is the table of mutants generated for the class:

Mutation Operator	**Original code**	**Mutated code**	**Killed/Survived**
Arithmetic Operator	internal static readonly int MaxCapacity =	internal static readonly int MaxCapacity = (Environment.ProcessorCount / 2) - 1;	Killed

	(Environment.ProcessorCount * 2) - 1;		
Block Statement Removal	_createFunc = createFunc;	{}	Killed
Equality Operator	(ref _fastItem, null, item) != item)	(ref _fastItem, null, item) == item)	Killed
Negate Condition	if (_items.TryDequeue(out item))	if (!(_items.TryDequeue(out item)))	Killed
Negate Condition	if (Interlocked.Increment(ref _numItems) <= MaxCapacity)	if (!(Interlocked.Increment(ref _numItems) <= MaxCapacity))	Killed
Statement Removal	return;	;	Killed
Conditional Boundaries	if (item == null || Interlocked.CompareExchange)	if (item != null || Interlocked.CompareExchange)	Killed
Statement Removal	_items.Enqueue(obj);	;	Killed
Negate Condition	if (!_returnFunc(obj))	if (_returnFunc(obj))	Killed
Statement Removal	Interlocked.Decrement(ref _numItems);	;	Survived

Analysis of the Surviving Mutant Case:

- **Test**: Add a test case to verify that _numItems is decremented correctly when an object is returned to the pool.
- **Test Input**: Create an object pool, get an object from the pool, and then return it to the pool.
- **Oracle**: Assert that _numItems is 0 after the object is returned.

To provide a detailed analysis, I answered the following questions from the table above:

1) How many mutants are killed? How many mutants survived?

- Total Mutants: 28
    - Killed Mutants: 14
    - Survived Mutants: 2

2) Are there any duplicate mutants?

- Yes, there are duplicate mutants in this analysis.
    - Specifically, the statement removal mutants (e.g., removing return;, _items.Enqueue(obj);, and Interlocked.Decrement(ref _numItems);) can be considered duplicates because they represent the same mutation operator applied in different cases.
    - While each of these mutants targets a different statement, they use the same mutation logic (removing the code line), which makes them functionally similar and, therefore, duplicates in terms of mutation testing.
    - This duplication could affect the effectiveness of the test suite by adding redundant testing of similar logic, without providing new insights into coverage gaps.

3) What are the challenges involved?

- Duplicate mutants add unwanted cases, making it harder to measure the test suite's effectiveness accurately. A manual review is needed to identify and exclude them.
- Surviving mutants are tough to catch due to hidden factors, while timeouts suggest performance issues that require more efficient tests or better time management.

4) Calculate and report the mutation score/effectiveness for your particular mutants.

- Mutation Score = (14 / 17) x 100= 82.36%

5) Does it affect the results?

- Yes, the presence of duplicate mutants could have changed the initial mutation score, making the test suite appear slightly more effective than it actually is in terms of unique logic coverage.

6) What do these results say about the effectiveness of the test suite?

- The 82.36% mutation score still indicates a fairly strong test suite, but the identification of duplicate mutants suggests there may be redundancy in test coverage.
- The surviving mutants and timeout highlight potential weaknesses, especially in testing internal state changes and performance-heavy scenarios.
- The effectiveness of the test suite can be improved by focusing on edge cases, optimizing tests for longer time, and reducing redundant test cases that do not add value.

7) Are there any cases with No Coverage?

- No. There are no reported cases with "No Coverage" in this analysis. All generated mutants were either killed or survived, indicating that all code areas under mutation were covered by the existing test cases.

3.6.2 PolicyBuilder.OrSyntax.cs

Here are the results of the class:

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
PolicyBuilder.OrSyntax.cs	81.48	91.67	22	2	0	3	13	0	9	22	5	49

Below is the table of mutants generated for the class:

Mutation operator	Original code	Mutated code	Killed/Survived
Statement Removal	ExceptionPredicates.Add(exception => exception is TException ? exception : null);	;	Killed
Conditional Expression	ExceptionPredicates.Add(exception => exception is TException ? exception : null);	ExceptionPredicates.Add(exception => (true?exception :null));	Killed
Conditional Expression	ExceptionPredicates.Add(exception => exception is TException ? exception : null);	ExceptionPredicates.Add(exception => (false?exception :null));	Killed
Equality Operator	if (matchedInAggregate != null)	if (matchedInAggregate == null)	Killed
Block Statement Removal	return matchedInAggregate;	{}	Killed
Negate Condition	(current == null)	(current != null)	Killed
Negate Condition	if (predicate(current))	if (!(predicate(current)))	Killed
Conditional Expression	ExceptionPredicates.Add(exception => exception is TException ? exception : null);	ExceptionPredicates.Add(exception => (true?exception :null));	Survived
Statement Removal	ResultPredicates.Add(predicate);	;	Killed

Logical Operator	OrResult(r => (!Equals(r, default(TResult)) && r.Equals(result)) || (Equals(r, default(TResult)) && Equals(result, default(TResult))));	OrResult(r => (!Equals(r, default(TResult)) || r.Equals(result)) || (Equals(r, default(TResult)) && Equals(result, default(TResult))));	Killed

Addressing Survived Mutants:

- **Test**: Add a test case to verify ExceptionPredicates correctly identifies TException.
- **Test Input**: Pass exceptions to ExceptionPredicates.
- **Oracle**: Assert that only TException is returned and the mutated condition (always returning true) causes the test to fail.

To provide a detailed analysis, I answered the following questions from the table above:

1) How many mutants are killed? How many mutants survived?

- Total Mutants Generated: 49
- Killed Mutants: 22
- Survived Mutants: 2

2) Are there any duplicate mutants?

- Yes, there are duplicate mutants in this analysis:
    - Statement Removal (;): Multiple mutants result from removing different statements in the code, creating similar effects across various lines.
    - Conditional Expression (ExceptionPredicates.Add(exception => (true? exception : null))): This mutation appears in more than one location, leading to redundant test scenarios.
    - These duplicates can change the total mutant count, making the test suite appear more comprehensive than it is in terms of unique logic coverage.

3) What are the challenges involved?

- Testing complex conditions is hard, needing diverse inputs. "No Coverage" mutants reveal untested code gaps.
- Equivalent Mutants: Finding them is manual and time-consuming, requiring a deep understanding of the code's logic.

4) Calculate and report the mutation score/effectiveness for your particular mutants.

- Mutation Score = (22 / 36) x 100 = 61.11%

5) Does it affect the results?

- Yes, the mutation score of 61.11% indicates that the test suite is only moderately effective, as it fails to kill nearly 39% of the mutants. The presence of duplicate mutants could also change the effectiveness of the test suite. The surviving mutants reveal gaps in test coverage, particularly in handling conditional expressions, which affects the accuracy of the test suite evaluation.

6) What do these results say about the effectiveness of the test suite?

- The test suite for PolicyBuilder.OrSyntax.cs has a moderate level of effectiveness. While it successfully kills over 60% of the mutants, it struggles with complex conditional expressions and logic changes. The results suggest that additional tests are needed to cover edge cases and improve detection of logical errors, especially those involving conditional logic.

7) Are there any cases with No Coverage?

- Yes, there are 3 mutants with "No Coverage." These mutants were not executed by any test case, indicating that certain branches or parts of the code were not reached during testing. This highlights clear gaps in the test suite that need to be addressed by adding new tests.

3.6.3 AsyncTimeoutEngine.cs

Here are the results of the class:

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
AsyncTimeoutEngine.cs	76.47	76.47	13	4	0	0	9	0	0	13	4	26

Below is the table of mutants generated for the class:

Mutation Operator	Original code	Mutated code	Killed/Survived
Negate Condition	if (timeoutStrategy == TimeoutStrategy.Optimistic)	if (timeoutStrategy != TimeoutStrategy.Optimistic)	Killed
Negate Condition	if (ex is OperationCanceledException && timeoutCancellationTokenSource.IsCancellationRequested)	if (!(ex is OperationCanceledException && timeoutCancellationTokenSource.IsCancellationRequested))	Killed
Logical Operator	if (ex is OperationCanceledException && timeoutCancellationTokenSource.IsCancellationRequested)	if (ex is OperationCanceledException || timeoutCancellationTokenSource.IsCancellationRequested)	Killed
Statement Removal	throw;	;	Killed
Conditional Boundaries	if (!combinedTokenSource.IsCancellationRequested && timeoutCancellationTokenSource.IsCancellationRequested)	if (combinedTokenSource.IsCancellationRequested && timeoutCancellationTokenSource.IsCancellationRequested)	Killed
Block Statement Removal	combinedTokenSource.Cancel();	{}	Killed
Negate Condition	if (!(!combinedTokenSource.IsCancellationRequested &&	if (!combinedTokenSource.IsCancellationRequested && timeoutCancellationTokenSo	Killed

	timeoutCancellationTokenSource.IsCancellationRequested))	urce.IsCancellationRequested)	
Logical Operator	if (!combinedTokenSource.IsCancellationRequested && timeoutCancellationTokenSource.IsCancellationRequested)	if (!combinedTokenSource.IsCancellationRequested || timeoutCancellationTokenSource.IsCancellationRequested)	Survived
String Replacement	throw new TimeoutRejectedException("The delegate executed asynchronously through TimeoutPolicy did not complete within the timeout.", ex);	throw new TimeoutRejectedException(" ", ex);	Survived
Parameter Change	useSynchronizationContext: false);	useSynchronizationContext: true);	Survived

Addressing Survived Mutants:

1	Test: Add a test to handle logical operator mutation. Test Input: Provide inputs that make both conditions either true or false. Oracle: Assert that the conditions are evaluated correctly under both scenarios.
2	Test: Add a test to handle string replacement mutation. Test Input: Trigger an exception with a specific message. Oracle: Assert that the exact exception message is returned as expected.
3	Test: Add a test to handle parameter change mutation. Test Input: Set useSynchronizationContext to true and execute the function. Oracle: Assert that the behavior changes accordingly when useSynchronizationContext is true.

To provide a detailed analysis, I answered the following questions from my the table above:

1) How many mutants are killed? How many mutants survived?

- Total Mutants: 26
- Killed Mutants: 13
- Survived Mutants: 4

2) Are there any duplicate mutants?

- Yes, there are duplicate mutants in the analysis:
    - Multiple mutations involve the logical operator changes, leading to similar testing scenarios.
    - These duplicates do not contribute unique insights and could skew the interpretation of test effectiveness.

3) What are the challenges involved?

- Logical Operators: It's hard to create tests that check all true/false combinations.
- String Replacement: Matching exact exception messages can be tricky due to small differences.

4) Calculate and report the mutation score/effectiveness for your particular mutants.

- Mutation score = (13 / 17)) * 100 = 76.47 %

5) Does it affect the results?

- Yes, the revised mutation score of 76.47% indicates that the test suite is more effective than initially thought. This higher score shows that the suite can handle most mutations, though some logical conditions and parameter-specific changes still need additional tests to be fully covered.

6) What do these results say about the effectiveness of the test suite?

- The 76.47% mutation score suggests that the test suite is fairly effective. It manages to catch most logical and parameter mutations but still misses some edge cases, such as complex logical conditions and specific string replacements. To improve further, the suite should include more targeted tests for parameter variations and specific error messages.

7) Are there any cases with No Coverage?

- No, there are 0 cases marked as "No Coverage," indicating that all mutants were at least executed by the test suite, even if they were not killed.

4. Comparative Analysis (and Conclusion)

In this exercise, we analyzed two different open-source projects, JSoup (Java) and Polly (C#), using PITest and Stryker. Our goal was to evaluate the comprehensiveness of their test suites by introducing controlled mutations in the source code and gathering insights on how well the tests detected them.

Key differences:

Metric	**JSoup**	**Polly**
Lines of Code	32.7k	68.7k
Test Framework	JUnit 5	xUnit
Total Tests	1364	2769
Test Strength	81%	69%
Execution Time	28 hours	2 hours

Both PITest and Stryker were effective in revealing gaps in test suites, with some key differences:

- Execution Time: Stryker was significantly faster (2 hours) compared to PITest (28 hours). This may be due to the nature of the projects and the tools themselves, with Stryker being more optimized for .NET environments.
- Test Suite Effectiveness: Both projects demonstrated high line coverage, but mutation coverage was less than we expected. For such a huge open-source project, we would expect the score to be significantly higher. JSOUP had a higher overall mutation coverage but also faced more runtime challenges.

In terms of the two tools, we observed the following differences:

- PITest was easy to set up as it was integrated with Maven allowing us to choose different configurations. The learning curve was slightly moderate compared to Stryker.
- Stryker was also easy to set up but the main challenge was ensuring the dependencies match for it to run with no errors.
- While PITest did include experimental mutation operators, Stryker was straightforward with its default operators making it easier to understand the report.
- Stryker provides real-time feedback during testing making it easy for us to monitor the process and get an expected run time while with PITest, we had no idea when the test run would complete.
- In terms of cost, PITest is time-consuming. However, the report is highly detailed.

5. Project Summary

Below is a summary of the individual analysis:

5.1 Dhruv

- PITest Analysis
    - HtmlTreeBuilderState.java
- Stryker Analysis
    - ObjectPool.cs
    - AsyncTimeoutEngine.cs

5.2 Vraj

- PITest Analysis
    - HttpConnection.java
    - SimpleBufferedInput.java
- Stryker Analysis
    - PolicyBuilder.OrSyntax.cs

(References have been included in terms of hyperlinks throughout the report)